Coding & Solution

Date	15 March 2026
Team ID	unm293200909
Project Name	FitBuddy - AI Fitness Plan Generator using Gemini Models
Name	Kavyasri.A
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of the implemented code and the overall solution. FitBuddy demonstrates working functionality, clean code practices, and alignment with the defined requirements.

Instructions:

1. Code is well-structured into separate modules and follows FastAPI best practices.
2. The solution addresses the core problem statement and implements the key functional requirements.
3. All source files, templates and setup instructions needed to run the application are included.
4. The code repository link is given in the table below.

Solution Summary

Field	Details
Repository Link / URL	[Paste your GitHub link here]
Programming Language(s)	Python, HTML, CSS (Jinja2 templates)
Framework(s) Used	FastAPI, Uvicorn, Jinja2, SQLAlchemy; Google Gemini API (Gemini 1.5 Pro and Gemini Flash)
Key Features Implemented	- Personalised 7-day workout plan using Gemini 1.5 Pro - Feedback-based plan update (e.g. more cardio, more rest days) - Goal-based nutrition / recovery tip using Gemini Flash - User details and plans stored in SQLite using SQLAlchemy - Admin dashboard (/view-all-users) to view all users and plans - HTML interface (Jinja2) and API endpoints (/docs)
Pending / Incomplete Features	All core features in the project scope are implemented. Possible future improvements: user login, progress tracking and downloadable plans.
Setup / Run Instructions	1. Install Python and pip. 2. Create a virtual environment: python -m venv fitbuddy-env and activate it. 3. Install packages: pip install fastapi uvicorn jinja2 sqlalchemy python-multipart google-generativeai 4. Add your Gemini API key as an environment variable. 5. Run: uvicorn app.main:app --reload 6. Open http://127.0.0.1:8000 (API docs at /docs).

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	**Yes**
2	Meaningful variable and function names are used	**Yes**
3	Code includes comments / documentation where necessary	**Yes**
4	Error handling is implemented for critical operations	**Yes**
5	The application runs without critical errors	**Yes**
6	Code is committed to a version control repository	**Yes**

Additional Notes / Comments:

The code is split into gemini_generator.py, gemini_flash_generator.py, updated_plan.py, database.py and routes.py, with templates index.html, result.html and all_users.html. The application runs locally with Uvicorn.

Work Assignment:

Coding Work Assigned	Assigned To
Set up environment, Gemini API key and project folders	Kavyasri.A
Write gemini_generator.py, gemini_flash_generator.py and updated_plan.py	Kavyasri.A
Write database.py (SQLAlchemy models and functions)	Kavyasri.A
Write routes.py and main application logic	Kavyasri.A
Create Jinja2 templates: index.html, result.html, all_users.html	Kavyasri.A
Test all scenarios and push the code to GitHub	Kavyasri.A